

2020년 1학기 C프로그래밍 2반 (40121-2)

실기시험3 풀이

6/19 (금요일) 10:00~11:50

다음에 제시된 풀이는 문제를 해결할 수 있는 여러 알고리즘들 한 가지일 뿐이다. 다른 알고리즘에 의해서도 문제가 요구하는 기능을 수행하도록 프로그램을 작성할 수 있다.

```
#include <stdio.h>

#define CR_BYTE 0x0D
#define LF_BYTE 0x0A

int getOneByte(FILE *fp, unsigned char *byte)
{
    if (fread(byte, 1, 1, fp) < 1)
        return 0;

    return 1;
}

int putBytes(FILE *fp, const unsigned char *bytes, int count)
{
    if (fwrite(bytes, 1, count, fp) != count)
        return 0;

    return 1;
}

char checkFile(char *fname)
{
    FILE *fp;
    char mode = '!';
    unsigned char inByte;

    if ((fp = fopen(fname, "rb")) == NULL)
    {
        fprintf(stderr, "checkFile: cannot open file %s\n", fname);
        return '!';
    }

    while (getOneByte(fp, &inByte))
    {
        switch (mode)
        {
            case '!':
                if (inByte == CR_BYTE)
                {
                    if (!getOneByte(fp, &inByte))
                    {
                        fclose(fp);
                        return '?';
                    }
                    if(inByte == LF_BYTE)
                        mode = 'W'; // 처음 CR/LF를 만나면 윈도우즈 파일로 잠정 판정
                    else
                        mode = 'A'; // 처음 CR/(LF아님)을 만나면 애플 OS 파일로 잠정 판정
                }
                else if (inByte == LF_BYTE)
                {
                    mode = 'L'; // 처음 LF를 만나면 리눅스 파일로 잠정 판정
                    break;
                }
            case 'W':
                if (inByte == CR_BYTE)
                {
                    if (!getOneByte(fp, &inByte))
                    {
                        fclose(fp);
                    }
                }
            case 'L':
                if (inByte == CR_BYTE)
                {
                    if (!getOneByte(fp, &inByte))
                    {
                        fclose(fp);
                    }
                }
            case 'A':
                if (inByte == LF_BYTE)
                {
                    if (!getOneByte(fp, &inByte))
                    {
                        fclose(fp);
                    }
                }
        }
    }
}
```

```

        return '?';
    }
    if (inByte == LF_BYTE)
        ; // 또 CR/LF를 만나면 윈도우즈 파일 판정 유지
    else
    {
        fclose(fp);
        return '?';
    }
}
else if (inByte == LF_BYTE)
{
    fclose(fp);
    return '?';
}
break;
case 'L':
    if (inByte == CR_BYTE)
        return '?';
    else if (inByte == LF_BYTE)
        ; // 또 LF를 만나면 리눅스 파일 판정 유지
    break;
case 'A':
    if (inByte == CR_BYTE)
        ; // 또 CR을 만나면 애플OS 파일 판정 유지
    else if (inByte == LF_BYTE)
    {
        fclose(fp);
        return '?';
    }
    break;
}

fclose(fp);

return mode;
}

int main(int argc, char *argv[])
{
    FILE *fpin, *fpout;
    unsigned char inByte, trashInByte, checkByte, outBuf[2];
    unsigned int size;
    char mode, curMode;

    if (argc < 4)
    {
        fprintf(stderr, "Usage: %s -W|L|A input_file output_file\n", argv[0]);
        return -1;
    }

    mode = argv[1][1];
    if (!(mode == 'W' || mode == 'L' || mode == 'A'))
    {
        fprintf(stderr, "%s: wrong OS indicator %c (only W, L, A accepted)\n",
            argv[0], mode);
        return -1;
    }

    if ((curMode = checkFile(argv[2])) == mode)
    {
        fprintf(stderr, "%s: the input file in the wanted '%c' format. "
            "No output file is generated!\n", argv[0], mode);
        return 0;
    }
    else if (curMode == '*')
    {
        fprintf(stderr, "%s: cannot open file %s\n", argv[0], argv[2]);
        return -1;
    }
    else if (curMode == '?')

```

```

{
    fprintf(stderr, "%s: the file %s corrupted or not a valid ASCII text file\n",
        argv[0], argv[2]);
    return -1;
}
else if (curMode == '!')
{
    fprintf(stderr, "%s: the file %s has no newline. "
        "No output file is generated!\n", argv[0], argv[2]);
    return 0;
}

fpin = fopen(argv[2], "rb");
if ((fpout = fopen(argv[3], "wb")) == NULL)
{
    fprintf(stderr, "%s: cannot open output file %s\n", argv[0], argv[3]);
    return -1;
}

while (getOneByte(fpin, &inByte))
{
    if (mode == 'W') // curMode == 'L' || curMode == 'A'
    {
        checkByte = (curMode == 'L') ? LF_BYTE : CR_BYTE;
        if (inByte == checkByte)
        {
            outBuf[0] = CR_BYTE, outBuf[1] = LF_BYTE;
            size = 2;
        }
        else // an input charater not related to a new line
        {
            outBuf[0] = inByte;
            size = 1;
        }
    }
    else if (mode == 'L')
    {
        if (inByte == CR_BYTE) // curMode == 'W' || curMode == 'A'
            if (curMode == 'W')
                getOneByte(fpin, &inByte); // omit CR and read in LF
            else
                inByte = LF_BYTE;
        outBuf[0] = inByte;
        size = 1;
    }
    else // mode == 'A'
    {
        if (inByte == CR_BYTE) // curMode == 'W'
            getOneByte(fpin, &trashInByte); // omit LF
        else if (inByte == LF_BYTE) // curMode == 'L'
            inByte = CR_BYTE;
        outBuf[0] = inByte;
        size = 1;
    }

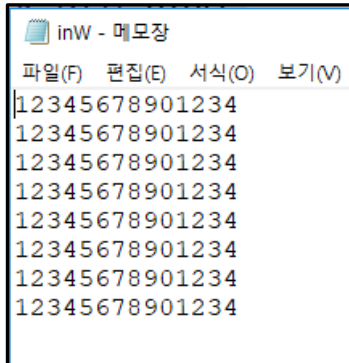
    if (!putBytes(fpout, outBuf, size))
    {
        fprintf(stderr, "%s: cannot write to %s\n", argv[0], argv[3]);
        return -1;
    }
}
fclose(fpin);
fclose(fpout);

return 0;
}

```

시행 예:

아래 시행 예는 문제의 지문에 나와 있는 시행 예인데, showfile 실행 결과까지 모두 보인 것이다. 먼저 프로그램을 검사하기 위해 만든 inW.txt 파일의 내용을 메모장을 사용해 보았다. 리눅스나 애플OS에서 만든 파일을 구할 수 없으므로 윈도우 파일을 리눅스 파일이나 애플OS 파일로 변환하여 시험에 사용하도록 하였다.



```

C:\ 명령 프롬프트

I:\Wtest>dir
I 드라이브의 볼륨: 새 볼륨
볼륨 일련 번호: 9C85-E01F

I:\Wtest 디렉터리

2019-11-24 오후 10:13 <DIR> .
2019-11-24 오후 10:13 <DIR> ..
2019-11-24 오후 09:56          41,472 convText.exe
2019-06-11 오후 10:56           128 inW.txt
2017-05-23 오전 03:06          11,685 jpeg.jpg
2019-11-24 오후 05:52          40,960 showfile.exe
                4개 파일              94,245 바이트
                2개 디렉터리  3,006,474,780,672 바이트 남음

I:\Wtest>convText
Usage: convText -W!L!A input_file output_file

I:\Wtest>convText -K inW.txt outK.txt
convText: wrong OS indicator K (only W, L, A accepted)

I:\Wtest>convText -W inW.txt outW.txt
convText: the input file in the wanted 'W' format. No output file is generated!

I:\Wtest>convText -L inW.txt outL.txt
  
```

명령 프롬프트

```
I:\Wtest>showfile inW.txt
000000: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||
000010: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||
000020: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||
000030: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||
000040: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||
000050: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||
000060: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||
000070: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||

I:\Wtest>showfile outL.txt
000000: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0A 31 || 12345678901234→1 ||
000010: 32 33 34 35 36 37 38 39 30 31 32 33 34 0A 31 32 || 2345678901234→12 ||
000020: 33 34 35 36 37 38 39 30 31 32 33 34 0A 31 32 33 || 345678901234→123 ||
000030: 34 35 36 37 38 39 30 31 32 33 34 0A 31 32 33 34 || 45678901234→1234 ||
000040: 35 36 37 38 39 30 31 32 33 34 0A 31 32 33 34 35 || 5678901234→12345 ||
000050: 36 37 38 39 30 31 32 33 34 0A 31 32 33 34 35 36 || 678901234→123456 ||
000060: 37 38 39 30 31 32 33 34 0A 31 32 33 34 35 36 37 || 78901234→1234567 ||
000070: 38 39 30 31 32 33 34 0A || 8901234→ ||

I:\Wtest>
```

아래 시행 예는 MS 윈도우즈, 리눅스, 애플 OS의 3가지 OS들 중 서로 다른 2가지 OS 간의 텍스트 파일 변환 6가지를 보인 것이다. (W→L, L→A, A→W, W→A, A→L, L→W의 6가지 변환)

명령 프롬프트

```
I:\Wtest>showfile inW.txt
000000: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||
000010: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||
000020: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||
000030: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||
000040: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||
000050: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||
000060: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||
000070: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A || 12345678901234↔ ||

I:\Wtest>convText -L inW.txt inL.txt

I:\Wtest>showfile inL.txt
000000: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0A 31 || 12345678901234→1 ||
000010: 32 33 34 35 36 37 38 39 30 31 32 33 34 0A 31 32 || 2345678901234→12 ||
000020: 33 34 35 36 37 38 39 30 31 32 33 34 0A 31 32 33 || 345678901234→123 ||
000030: 34 35 36 37 38 39 30 31 32 33 34 0A 31 32 33 34 || 45678901234→1234 ||
000040: 35 36 37 38 39 30 31 32 33 34 0A 31 32 33 34 35 || 5678901234→12345 ||
000050: 36 37 38 39 30 31 32 33 34 0A 31 32 33 34 35 36 || 678901234→123456 ||
000060: 37 38 39 30 31 32 33 34 0A 31 32 33 34 35 36 37 || 78901234→1234567 ||
000070: 38 39 30 31 32 33 34 0A || 8901234→ ||
```

CA 명령 프롬프트

```
I:\Wtest>convText -A inL.txt inA.txt
```

```
I:\Wtest>showfile inA.txt
```

```
000000: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 31 11 12345678901234<1 11
000010: 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 31 32 11 2345678901234<12 11
000020: 33 34 35 36 37 38 39 30 31 32 33 34 0D 31 32 33 11 345678901234<123 11
000030: 34 35 36 37 38 39 30 31 32 33 34 0D 31 32 33 34 11 45678901234<1234 11
000040: 35 36 37 38 39 30 31 32 33 34 0D 31 32 33 34 35 11 5678901234<12345 11
000050: 36 37 38 39 30 31 32 33 34 0D 31 32 33 34 35 36 11 678901234<123456 11
000060: 37 38 39 30 31 32 33 34 0D 31 32 33 34 35 36 37 11 78901234<1234567 11
000070: 38 39 30 31 32 33 34 0D 11 8901234< 11
```

CA 명령 프롬프트

```
I:\Wtest>convText -W inA.txt outW.txt
```

```
I:\Wtest>showfile outW.txt
```

```
000000: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A 11 12345678901234<1 11
000010: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A 11 12345678901234<12 11
000020: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A 11 12345678901234<123 11
000030: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A 11 12345678901234<1234 11
000040: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A 11 12345678901234<12345 11
000050: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A 11 12345678901234<123456 11
000060: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A 11 12345678901234<1234567 11
000070: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A 11 12345678901234<12345678 11
```

CA 명령 프롬프트

```
I:\Wtest>convText -A outW.txt outA.txt
```

```
I:\Wtest>showfile outA.txt
```

```
000000: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 31 11 12345678901234<1 11
000010: 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 31 32 11 2345678901234<12 11
000020: 33 34 35 36 37 38 39 30 31 32 33 34 0D 31 32 33 11 345678901234<123 11
000030: 34 35 36 37 38 39 30 31 32 33 34 0D 31 32 33 34 11 45678901234<1234 11
000040: 35 36 37 38 39 30 31 32 33 34 0D 31 32 33 34 35 11 5678901234<12345 11
000050: 36 37 38 39 30 31 32 33 34 0D 31 32 33 34 35 36 11 678901234<123456 11
000060: 37 38 39 30 31 32 33 34 0D 31 32 33 34 35 36 37 11 78901234<1234567 11
000070: 38 39 30 31 32 33 34 0D 11 8901234< 11
```

CA 명령 프롬프트

```
I:\Wtest>convText -L outA.txt outL.txt
```

```
I:\Wtest>showfile outL.txt
```

```
000000: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0A 31 11 12345678901234>1 11
000010: 32 33 34 35 36 37 38 39 30 31 32 33 34 0A 31 32 11 2345678901234>12 11
000020: 33 34 35 36 37 38 39 30 31 32 33 34 0A 31 32 33 11 345678901234>123 11
000030: 34 35 36 37 38 39 30 31 32 33 34 0A 31 32 33 34 11 45678901234>1234 11
000040: 35 36 37 38 39 30 31 32 33 34 0A 31 32 33 34 35 11 5678901234>12345 11
000050: 36 37 38 39 30 31 32 33 34 0A 31 32 33 34 35 36 11 678901234>123456 11
000060: 37 38 39 30 31 32 33 34 0A 31 32 33 34 35 36 37 11 78901234>1234567 11
000070: 38 39 30 31 32 33 34 0A 11 8901234> 11
```

```
CA: 명령 프롬프트
I:\Wtest>convText -W outL.txt outW2.txt

I:\Wtest>showfile outW2.txt
000000: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A !! 12345678901234↔ !!
000010: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A !! 12345678901234↔ !!
000020: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A !! 12345678901234↔ !!
000030: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A !! 12345678901234↔ !!
000040: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A !! 12345678901234↔ !!
000050: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A !! 12345678901234↔ !!
000060: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A !! 12345678901234↔ !!
000070: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 0D 0A !! 12345678901234↔ !!

I:\Wtest>
```

아래 시행 예는 이미 해당 OS의 텍스트 형식으로 되어 있는 경우 3가지에 대한 시행 예이다. (W->W, L->L, A->의 3가지)

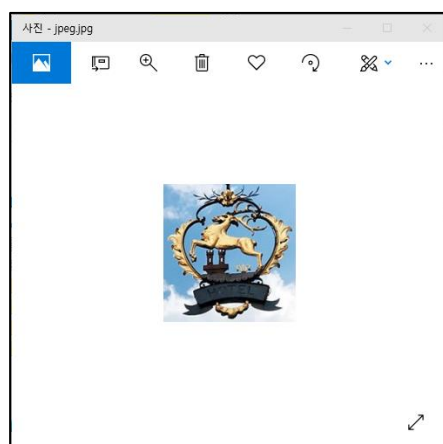
```
CA: 명령 프롬프트
I:\Wtest>convText -W inW.txt out.txt
convText: the input file in the wanted 'W' format. No output file is generated!

I:\Wtest>convText -L inL.txt out.txt
convText: the input file in the wanted 'L' format. No output file is generated!

I:\Wtest>convText -A inA.txt out.txt
convText: the input file in the wanted 'A' format. No output file is generated!

I:\Wtest>
```

아래 시행 예는 JPEG 파일을 MS 윈도우즈 텍스트 파일, 리눅스 텍스트 파일, 애플 OS 텍스트 파일 등으로 변환하려고 시도한 것이다. 우선 해당 JPEG 파일의 내용을 우선 사진 앱으로 보았다.



showfile을 사용하여 jpeg.jpg의 내용을 5줄만 살펴 보더라도, 단독의(즉 0x0D(CR)이 바로 앞에 먼저 나오지 않는) 0x0A(LF)가 등장하다가 단독의(즉 0x0A(LF)가 끝이여 뒤따라 나오지 않는) 0x0D(CR)이 등장하므로, 3가지 OS 중 어떤 것의 텍스트 파일도 아님을 알 수 있다. 따라서 아래 시행 예에서는 3가지 OS 중 어떤 OS의 텍스트 파일로도 변환되지 않으며 출력 파일도 만들어지지 않음을 볼 수 있다.

```

CA: 명령 프롬프트
I:\test>showfile jpeg.jpg 5
000000: FF D8 FF E0 00 10 4A 46 49 46 00 01 01 00 60 !!      JFIF      `  !!
000010: 00 60 00 00 FF DB 00 43 00 03 02 02 03 02 03 !!      C      !!
000020: 03 03 03 04 03 03 04 05 08 05 05 04 04 05 0A 07 !!      →  !!
000030: 07 06 08 0C 0A 0C 0C 0B 0A 0B 0B 0D 0E 12 10 0D !!      →  →  ←  ←  !!
000040: 0E 11 0E 0B 0B 10 16 10 11 13 14 15 15 15 0C 0F !!      !!
Type ENTER to continue: .

I:\test>convText -W jpeg.jpg out.txt
convText: the file jpeg.jpg corrupted or not a valid ASCII text file

I:\test>convText -L jpeg.jpg out.txt
convText: the file jpeg.jpg corrupted or not a valid ASCII text file

I:\test>convText -A jpeg.jpg out.txt
convText: the file jpeg.jpg corrupted or not a valid ASCII text file

I:\test>

```

아래 시행 예는 개행문자가 없는 텍스트 파일을 입력파일로 사용하는 경우, 3가지 OS 중 어떤 경우에도 변환할 필요가 없기 때문에 출력파일이 생성되지 않는 예이다. (이는 채점 범위에 포함되지 않는 사항임)

```

CA: 명령 프롬프트
I:\test>showfile no_newline.txt
000000: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 31 32 !! 1234567890123412 !!
000010: 33 34 35 36 37 38 39 30 31 32 33 34 31 32 33 34 !! 3456789012341234 !!
000020: 35 36 37 38 39 30 31 32 33 34 31 32 33 34 35 36 !! 5678901234123456 !!
000030: 37 38 39 30 31 32 33 34 31 32 33 34 35 36 37 38 !! 7890123412345678 !!
000040: 39 30 31 32 33 34 31 32 33 34 35 36 37 38 39 30 !! 9012341234567890 !!
000050: 31 32 33 34 31 32 33 34 35 36 37 38 39 30 31 32 !! 1234123456789012 !!
000060: 33 34 31 32 33 34 35 36 37 38 39 30 31 32 33 34 !! 3412345678901234 !!

I:\test>convText -W no_newline.txt out.txt
convText: the file no_newline.txt has no newline. No output file is generated!

I:\test>convText -L no_newline.txt out.txt
convText: the file no_newline.txt has no newline. No output file is generated!

I:\test>convText -A no_newline.txt out.txt
convText: the file no_newline.txt has no newline. No output file is generated!

I:\test>

```

끝.